Laboratorium 4

Modelowanie Układów Dynamicznych

Kamil Kania

2026-10-27

Table of contents

1 Zadanie 1 - układ równań **1**
1.1 Punkty stałe i ich stabilność 1
1.2 Portet fazowy równania 2
1.3 Rozwiązanie numeryczne 3
1.4 Przejście do współrzędnych biegunowych 5
1.5 Rozwiązanie $\theta(t)$ oraz $r(t)$ 6

2 Oscylator Rayleigha **9**

1 Zadanie 1 - układ równań

Dany jest układ równań

$$\dot{x} = -2y - x\left[(x^2+y^2)^2 - 4(x^2+y^2) + 1\right] \tag{1}$$
$$\dot{y} = 2x - y\left[(x^2+y^2)^2 - 4(x^2+y^2) + 1\right] \tag{2}$$

1.1 Punkty stałe i ich stabilność

Wyznaczenie punktów stałych sprowadza się do rozwiązania układu równań

$$-2y - x\left[(x^2+y^2)^2 - 4(x^2+y^2) + 1\right] = 0 \tag{3}$$
$$2x - y\left[(x^2+y^2)^2 - 4(x^2+y^2) + 1\right] = 0 \tag{4}$$

Mnożąc pierwsze równanie przez y oraz drugie przez x i odejmując pierwsze od drugiego otrzymujemy wyrażenie

$$2(x^2+y^2) = 0,$$

które jest spełnione jedynie dla $x = y = 0$.

Następnie, budujemy macierz Jacobiego

$$J = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} & \frac{\partial f(x,y)}{\partial y} \\ \frac{\partial g(x,y)}{\partial x} & \frac{\partial g(x,y)}{\partial y} \end{bmatrix}$$

gdzie

$$f(x,y) = -2y - x\left[(x^2+y^2)^2 - 4(x^2+y^2) + 1\right]$$

oraz

$$g(x,y) = 2x - y\left[(x^2+y^2)^2 - 4(x^2+y^2) + 1\right]$$

.

Jedyne interesujące nas pochodne są w punktach $x = y = 0$. Z tego względu bardzo się one upraszczają, co prowadzi nas do postaci

$$J = \begin{bmatrix} -1 & -2 \\ 2 & -1 \end{bmatrix}$$

Problemem własnym jest równanie $(1+\lambda)^2 + 4 = 0$, którego rozwiązaniem to para

$$\lambda_1 = -1 - 2i, \quad \lambda_2 = -1 + 2i$$

Część rzeczywista rozwiązania jest ujemna, zatem punkt $(0,0)$ jest punktem stałym stabilnym.

1.2 Portet fazowy równania

```
x = np.linspace(-5, 5, 1000)
y = np.linspace(-5, 5, 1000)

X, Y = np.meshgrid(x, y)

R = (X ** 2 + Y ** 2)

X_DOT = -2 * Y - X * (R ** 2 - 4 * R + 1)
Y_DOT = 2 * X - Y * (R ** 2 - 4 * R + 1)

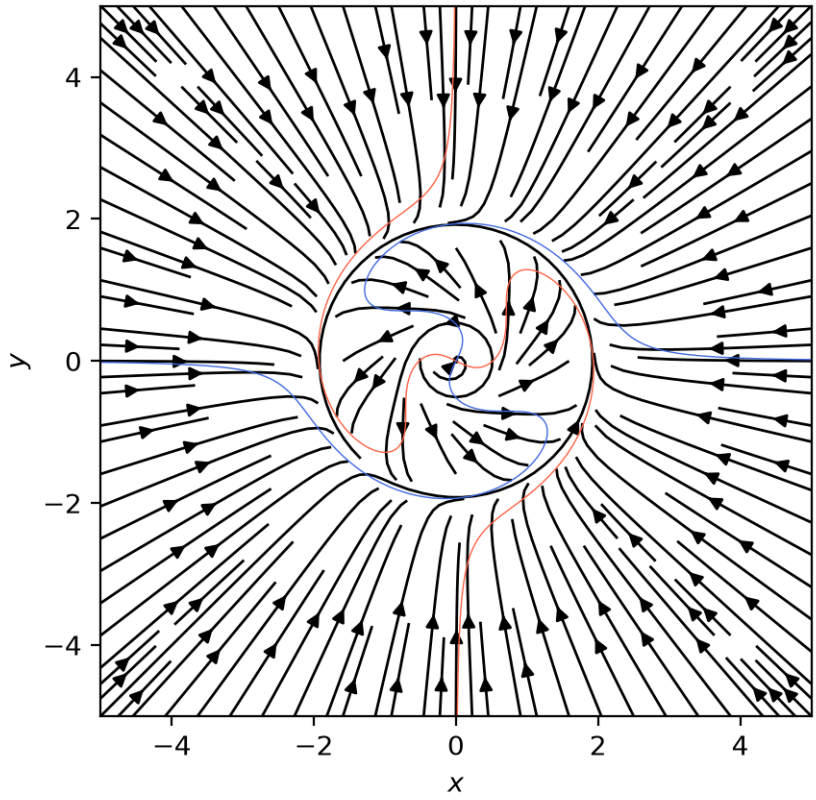
fig, ax = plt.subplots(dpi=200)

ax.streamplot(X,  Y, X_DOT, Y_DOT, linewidth=1, density=1.5, color='black')

ax.contour(X, Y, X_DOT, levels=[0], lw=0.5, colors='tomato')
ax.contour(X, Y, Y_DOT, levels=[0], lw=0.5, colors='royalblue')

ax.set_xlabel("$x$")
ax.set_ylabel("$y$")


plt.gca().set_aspect('equal')
```



1.3 Rozwiązanie numeryczne

```
t_min, t_max = 0, 10
t_span = [t_min, t_max]
t = np.linspace(t_min, t_max, 100)

def f(t, y):
    x, y = y

    x_dot = -2 * y - x * ((x ** 2 + y ** 2) ** 2 - 4 * (x ** 2 + y ** 2) + 1)
    y_dot = 2 * x - y * ((x ** 2 + y ** 2) ** 2 - 4 * (x ** 2 + y ** 2) + 1)

    return [x_dot, y_dot]

y0 = 0
x0_arr = [0.2, 0.4, 0.5175, 0.5180, 1.0, 1.8, 2.5]

solutions = np.zeros((len(x0_arr)), dtype=np.ndarray)

colors = plt.cm.viridis(np.linspace(0, 1, len(x0_arr)))

fig, (ax1, ax2) = plt.subplots(2, 1, dpi=200, sharex=True, constrained_layout=True)

for i, x0 in enumerate(x0_arr):
    sol = solve_ivp(
            f,
            t_span,
```

```
            [x0, y0],
            method="DOP853",
            rtol=1e-6,
            atol=1e-3,
            max_step=0.001
        )

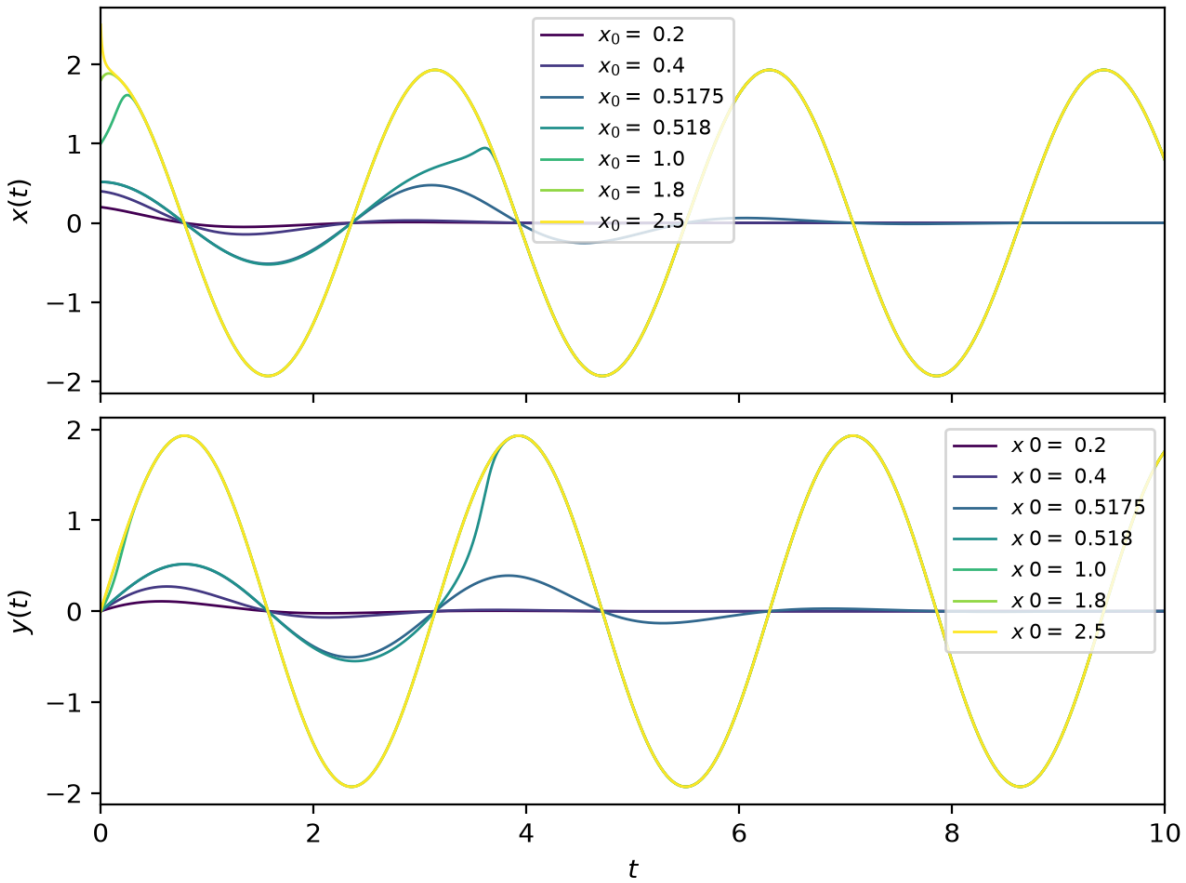
    solutions[i] = [sol.t, sol.y[0], sol.y[1]]

    ax1.plot(sol.t, sol.y[0], lw=1.0, label=f"$x_0 =$ {x0}", color=colors[i])

    ax1.set_ylabel("$x(t)$")
    ax1.legend(fontsize=8)

    ax2.plot(sol.t, sol.y[1], lw=1.0, label=f"$x_0 =$ {x0}", color=colors[i])

    ax2.set_xlabel("$t$")
    ax2.set_xlim(t_min, t_max)
    ax2.set_ylabel("$y(t)$")
    ax2.legend(fontsize=8)
```




```
fig, ax = plt.subplots(dpi=200)

for i, x0 in enumerate(x0_arr):
```

```
    res = solutions[i]

    x, y = res[1:]

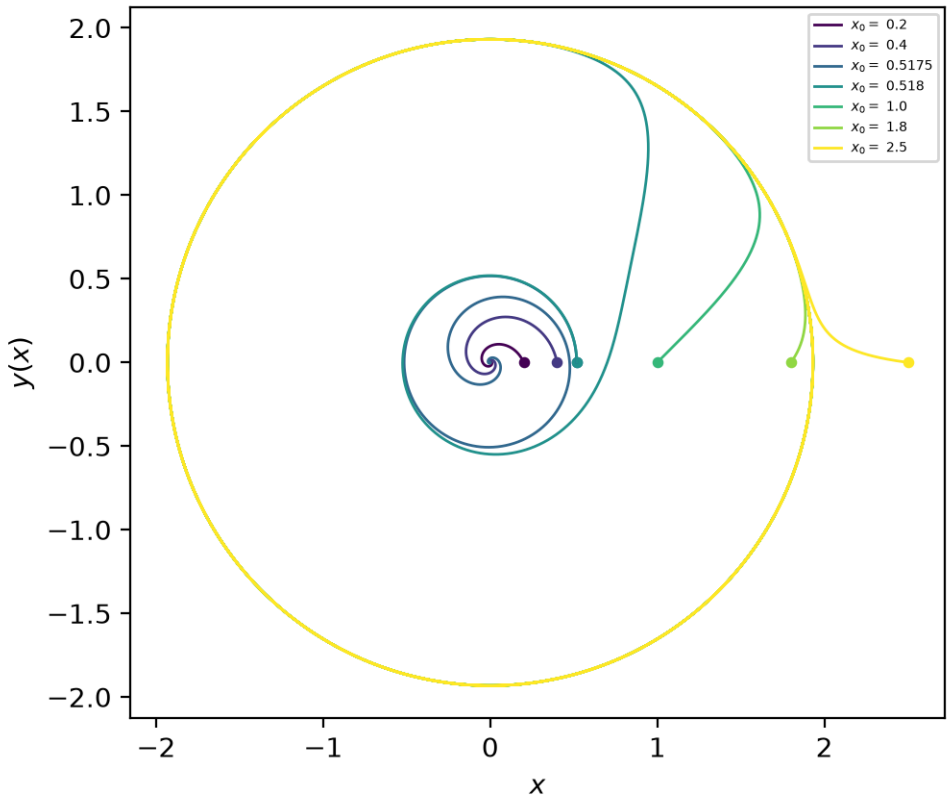
    ax.plot(x, y, color=colors[i], lw=1.0, label=f"$x_0 = $ {x0}")

    ax.scatter(x0, y0, color=colors[i], s=10)

plt.gca().set_aspect('equal')

ax.set_xlabel("$x$")
ax.set_ylabel("$y(x)$")

ax.legend(fontsize=5)
```



1.4 Przejście do współrzędnych biegunowych

Przejście do współrzędnych biegunowych sprowadza się do podstawień

$$x = r\cos(\theta) \tag{5}$$
$$y = r\sin(\theta) \tag{6}$$
$$r = \sqrt{x^2 + y^2} \tag{7}$$

Pochodne współrzędnych x i y można zapisać jako

$$\dot{x} = \dot{r}\cos(\theta) - r\dot{\theta}\sin(\theta) \tag{8}$$
$$\dot{y} = \dot{r}\sin(\theta) + r\dot{\theta}\cos(\theta) \tag{9}$$

Podstawiając te zależności do układu równań na $\dot{x}$ i $\dot{y}$ oraz mnożąc pierwsze wyrażenie przez $\sin(\theta)$, a drugie przez $\cos(\theta)$ otrzymujemy, że

$$\dot{\theta} = 2$$

Następnie, korzystając, że $\frac{d}{dt}(r^2) = 2r\dot{r} \Rightarrow r\dot{r} = x\dot{x} + y\dot{y}$ otrzymujemy, że

$$\dot{r} = -r(r^4 - 4r^2 + 1)$$

O stabilności rozwiązania decyduje zatem jedynie współrzędna radialna, gdyż układ wiruje ze stałą prędkością kątową.

Rozwiązujemy zatem równanie

$$-r(r^4 - 4r^2 + 1) = 0$$

z którego mamy dwa rozwiązania

$$r = 0 \quad \vee \quad r^4 - 4r^2 + 1 = 0$$

Jeżeli zatem promień wynosi $r_1 = 0$, to układ znajduje się w punkcie stałym. Pochodna wynosi wtedy $-4r^3 + 8r - 1 = -1 < 0$, zatem jest to punkt stabilny.

Drugie rozwiązanie to $r^4 - 4r^2 + 1 = 0$, gdzie podstawiając za $t = r^2$, otrzymujemy

$$t^2 - 4t + 1 = 0,$$

którego rozwiązaniem jest $t = 2 \pm \sqrt{3}$. Wracając do $r = \pm\sqrt{t}$ i uwzględniając, że interesują nas jedynie rozwiązania fizyczne, tj. $r \geq 0$, otrzymujemy ostatecznie rozwiązania $r_2 = \sqrt{2 - \sqrt{3}} \approx 0.51764$ oraz $r_3 = \sqrt{2 + \sqrt{3}} \approx 1.93$.

Patrząc na trajektorie w przestrzeni fazowej $(y(x), x)$ widać, że $r_2 \approx 0.51764$ jest wartością graniczną poniżej której rozwiązania zbiegają do $r_1 = 0$, natomiast dla $r_0 > r_2$ rozwiązania zbiegają do r_3. Rozwiązania powyżej r_3 również zbiegają do tej wartości, co widać bazując na trajektorii rozwiązania przy $r_0 = 2.5$.

Dla r_2 wartość pochodnej wynosi $1.86 > 0$, zatem jest to punkt stały niestabilny, natomiast dla r_3 pochodna jest ujemna $-25.86 < 0$, co świadczy o stabilności tego punktu stałego.

1.5 Rozwiązanie $\theta(t)$ oraz $r(t)$

Dla $\theta(t)$ można łatwo uzyskać rozwiązanie analityczne, gdyż prędkość kątowa jest wartością stałą. Oznacza to, że r. ruchu dla θ t

$$\theta(t) = 2t + \theta_0$$

Zatem obrót zawsze jest przeciwny do ruchu wskazówek zegara i nie zależy od promienia r.

```
r0_arr = [np.sqrt(y0 ** 2 + x0 ** 2) for x0 in x0_arr]
theta0 = 0

def f(t, y):
    theta, r = y

    d_theta = 2
    d_r = -r * (r ** 4 - 4 * r **2 + 1)

    return [d_theta, d_r]

solutions_polar = np.zeros((len(x0_arr)), dtype=np.ndarray)

fig, (ax1, ax2) = plt.subplots(2, 1, dpi=200, sharex=True, constrained_layout=True)

for i, r0 in enumerate(r0_arr):
    sol = solve_ivp(
            f,
            t_span,
            [theta0, r0],
            method="DOP853",
            rtol=1e-6,
            atol=1e-3,
            max_step=0.001
        )

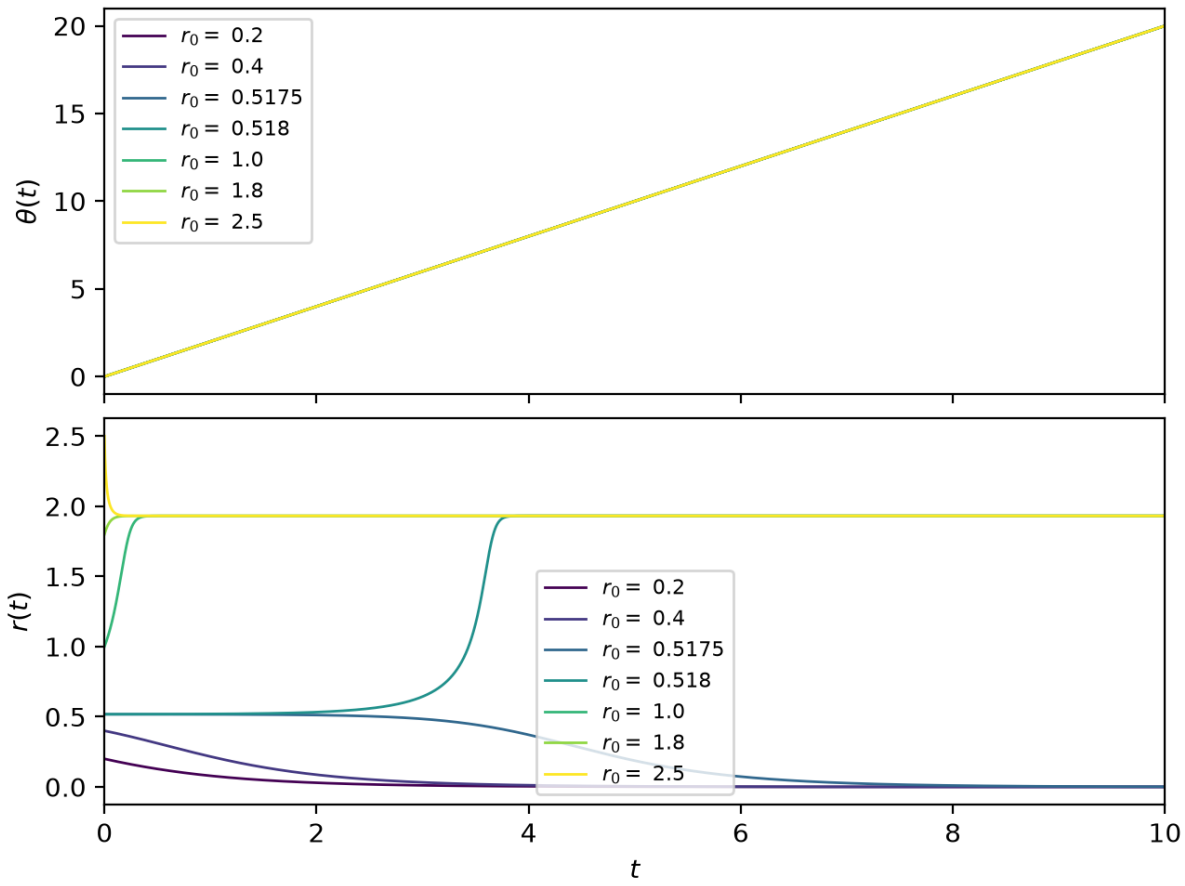
    solutions_polar[i] = [sol.t, sol.y[0], sol.y[1]]

    ax1.plot(sol.t, sol.y[0], lw=1.0, label=f"$r_0 =$ {r0}", color=colors[i])

    ax1.set_ylabel("$\\theta(t)$")
    ax1.legend(fontsize=8)

    ax2.plot(sol.t, sol.y[1], lw=1.0, label=f"$r_0=$ {r0}", color=colors[i])

    ax2.set_xlabel("$t$")
    ax2.set_xlim(t_min, t_max)
    ax2.set_ylabel("$r(t)$")
    ax2.legend(fontsize=8)
```




```
fig, ax = plt.subplots(dpi=200)

for i, r0 in enumerate(r0_arr):

    res = solutions_polar[i]

    t, theta, r = res

    x = r * np.cos(theta)
    y = r * np.sin(theta)

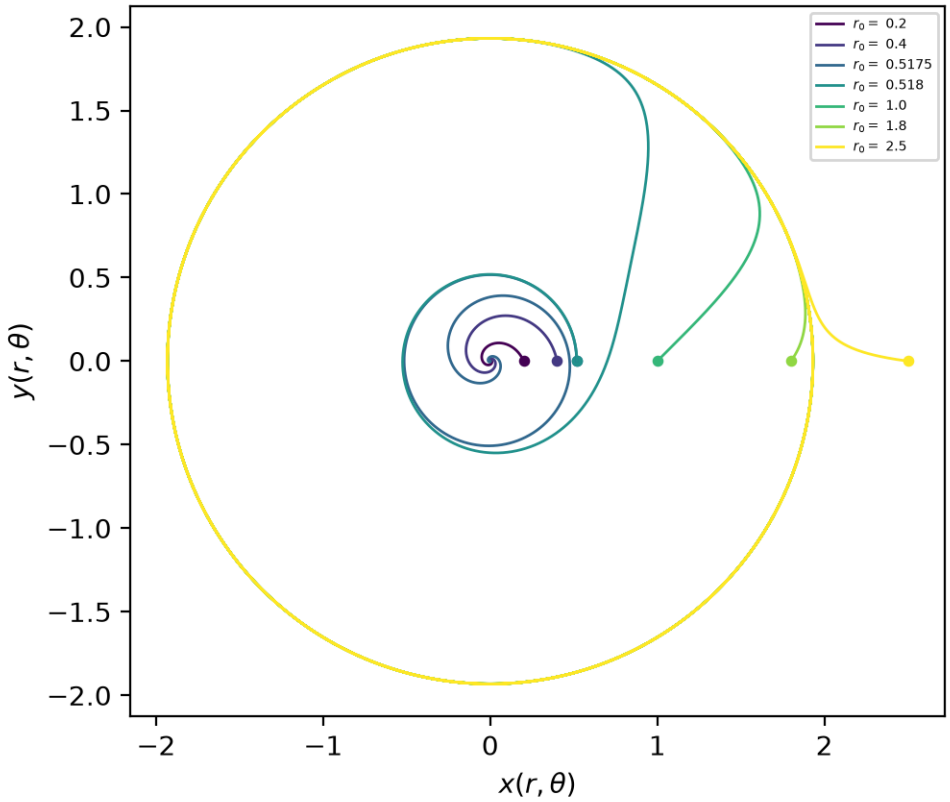
    ax.plot(x, y, color=colors[i], lw=1.0, label=f"$r_0 = $ {r0}")

    ax.scatter(r0, y0, color=colors[i], s=10)

plt.gca().set_aspect('equal')

ax.set_xlabel("$x(r, \\theta)$")
ax.set_ylabel("$y(r, \\theta)$")


ax.legend(fontsize=5)
```



2 Oscylator Rayleigha

Równanie oscylatora Rayleigha ma postać

$$\ddot{x} + \mu(\frac{1}{3}\dot{x}^2 - 1)\dot{x} + x = 0$$

Podstawiając za $y = \dot{x}$ oraz $\dot{y} = \ddot{x}$ otrzymujemy układ równań

$$\dot{x} = y \tag{10}$$

$$\dot{y} = -\mu(\frac{1}{3}y^2 - 1)y - x \tag{11}$$

Jedynym punktem stałym jest para $(x, y) = (0, 0)$.

Jacobian wynosi

$$J = \begin{bmatrix} 0 & 1 \\ -1 & \mu \end{bmatrix}$$

Dla punktu $(0, 0)$ mamy zatem, że $-\lambda(\mu - \lambda) + 1 = 0$, co dla $\mu = 2$ daje nam $(\lambda - 1)^2 = 0$.

Rozwiązanie $\lambda = 1 > 0$, zatem jest to punkt niestabilny.

```
x = np.linspace(-10, 10, 1000)
y = np.linspace(-5, 5, 1000)
```

```
X, Y = np.meshgrid(x, y)

mu = 2

X_DOT = Y
Y_DOT = -mu * (1 / 3 * Y ** 2 - 1) * Y - X

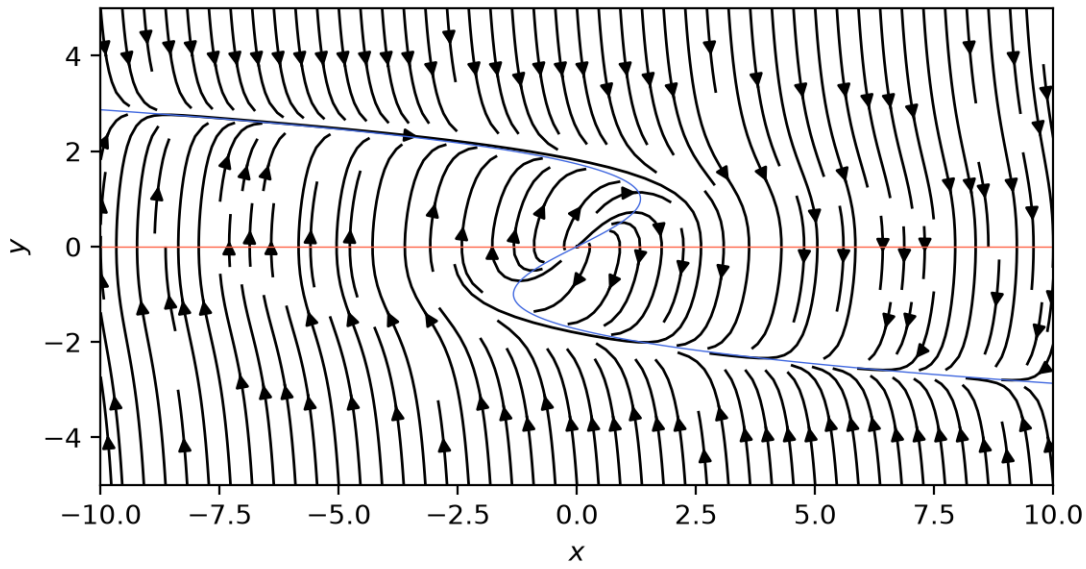
fig, ax = plt.subplots(dpi=200)

ax.streamplot(X,  Y, X_DOT, Y_DOT, linewidth=1, density=1.5, color='black')

ax.contour(X, Y, X_DOT, levels=[0], lw=0.5, colors='tomato')
ax.contour(X, Y, Y_DOT, levels=[0], lw=0.5, colors='royalblue')

ax.set_xlabel("$x$")
ax.set_ylabel("$y$")


plt.gca().set_aspect('equal')
```




```
t_min, t_max = 0, 20
t_span = [t_min, t_max]
t = np.linspace(t_min, t_max, 100)

x0_arr = [-5.0, -2.5, 0.0, 0.0, 2.5, 5.0]
y0_arr = [0.0, 0.0, 0.1, 0.0, -2.0, -4.0]

def f(t, y):
    x, y = y

    x_dot = y
    y_dot = -2 * (1 / 3 * y ** 2 - 1) * y - x

    return [x_dot, y_dot]
```

```
solutions = np.zeros((len(x0_arr)), dtype=np.ndarray)

colors = plt.cm.viridis(np.linspace(0, 1, len(x0_arr)))

fig, (ax1, ax2) = plt.subplots(2, 1, dpi=200, sharex=True, constrained_layout=True)

for i in range(len(x0_arr)):
    sol = solve_ivp(
            f,
            t_span,
            [x0_arr[i], y0_arr[i]],
            method="DOP853",
            rtol=1e-6,
            atol=1e-3,
            max_step=0.001
        )

    solutions[i] = [sol.t, sol.y[0], sol.y[1]]

    ax1.plot(sol.t, sol.y[0], lw=1.0, label=f"$(x_0,y_0) = $ {x0_arr[i], y0_arr[i]}", color=

    ax1.set_ylabel("$x(t)$")
    ax1.legend(fontsize=8)

    ax2.plot(sol.t, sol.y[1], lw=1.0, label=f"$(x_0,y_0) = $ {x0_arr[i], y0_arr[i]}", color=

    ax2.set_xlabel("$t$")
    ax2.set_xlim(t_min, t_max)
    ax2.set_ylabel("$y(t)$")
    ax2.legend(fontsize=8)


fig, ax = plt.subplots(dpi=200)

for i, x0 in enumerate(x0_arr):

    res = solutions[i]

    x, y = res[1:]

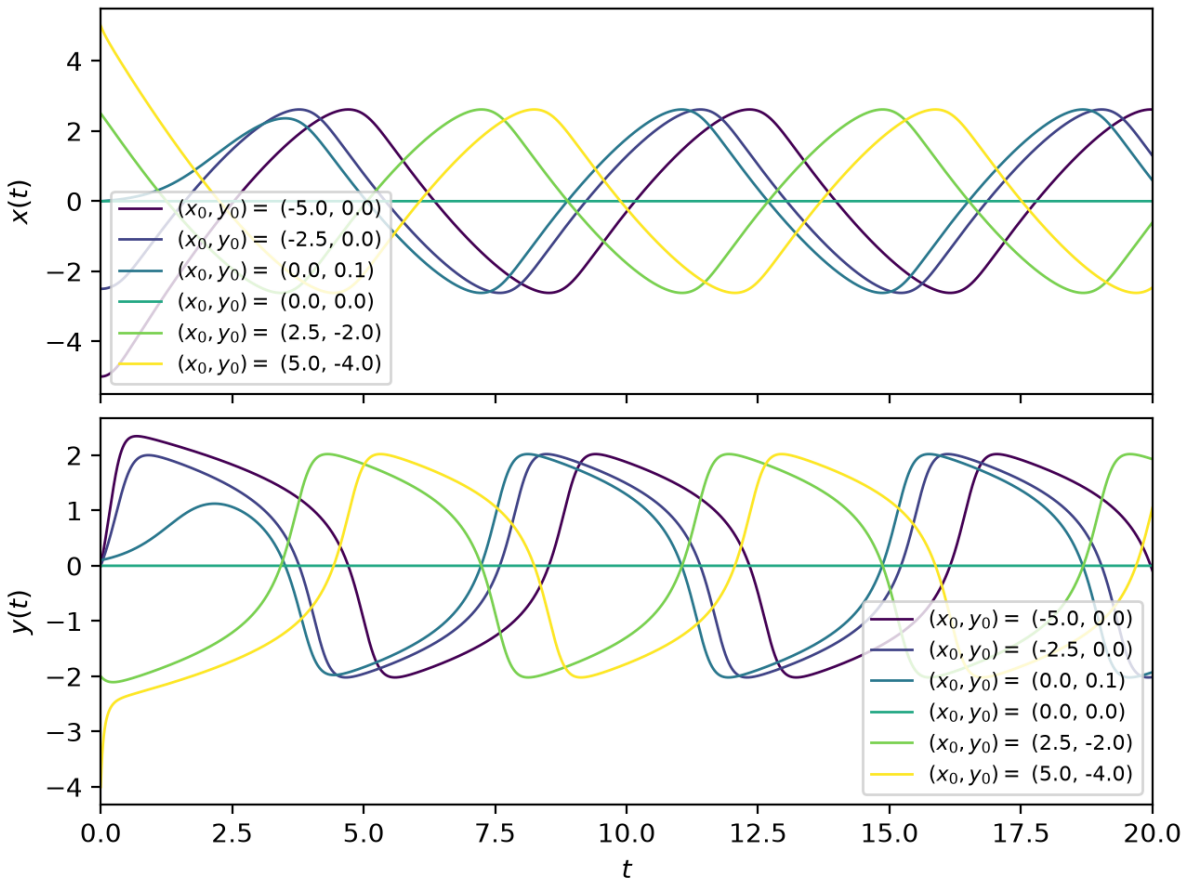
    ax.plot(x, y, color=colors[i], lw=1.0, label=f"$(x_0,y_0) = $ {x0_arr[i], y0_arr[i]}")

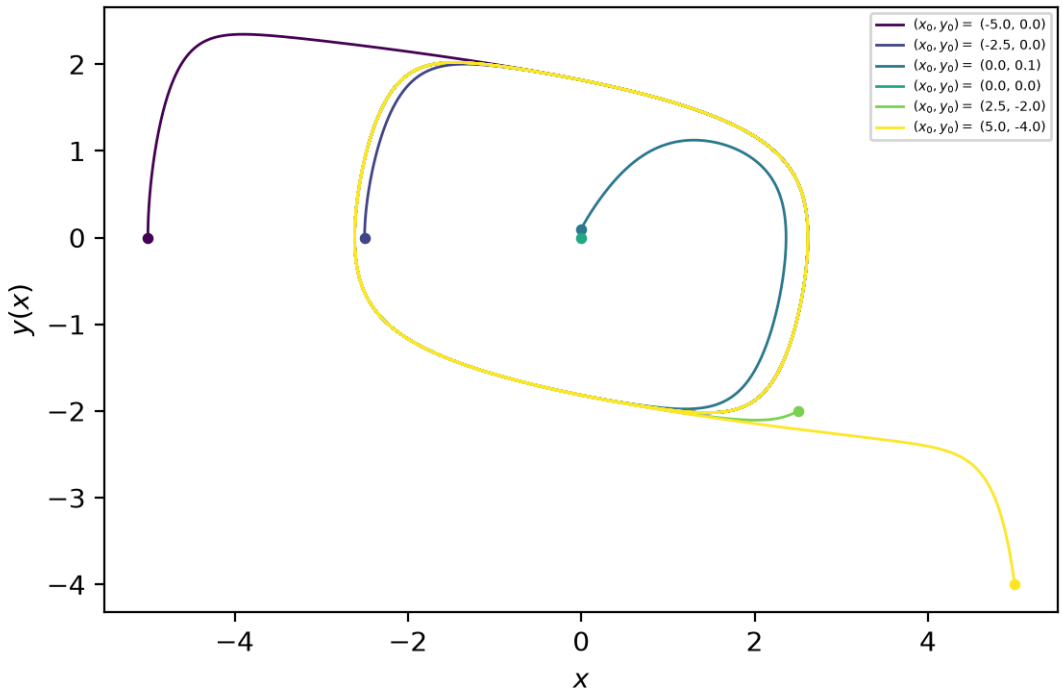
    ax.scatter(x0_arr[i], y0_arr[i], color=colors[i], s=10)

plt.gca().set_aspect('equal')

ax.set_xlabel("$x$")
ax.set_ylabel("$y(x)$")

ax.legend(fontsize=5)
```


x(t)
y(t)
t
(x0, y0) = (-5.0, 0.0)
(x0, y0) = (-2.5, 0.0)
(x0, y0) = (0.0, 0.1)
(x0, y0) = (0.0, 0.0)
(x0, y0) = (2.5, -2.0)
(x0, y0) = (5.0, -4.0)


y(x)
x
(x0, y0) = (-5.0, 0.0)
(x0, y0) = (-2.5, 0.0)
(x0, y0) = (0.0, 0.1)
(x0, y0) = (0.0, 0.0)
(x0, y0) = (2.5, -2.0)
(x0, y0) = (5.0, -4.0)